

# ExtractAnalyzeDocumentsMultimodalLLMs

May 19, 2026

## 1 Lab 7: Extracting and Analyzing Document Data with Multimodal LLMs

```
[1]: import os
import json
import base64
import pandas as pd
from openai import OpenAI
from IPython.display import display, Markdown, Image as IPIImage

# === OpenAI Client Setup ===
client = OpenAI()
```

Throughout this unit, you have explored how large language models work, their limitations, and how to build applications that use them. You have practiced making API calls, managing conversation history, engineering prompts, working with multimodal inputs, and extracting structured data from images. In this lab, you will apply several of those skills together to build a document extraction and analysis pipeline using a multimodal LLM.

You will complete the following tasks:

1. Load and inspect the sample documents:
  - Load a set of expense document images and convert them to base64 format for API use
  - Visually inspect the documents to understand their structure and variability
2. Design an extraction prompt and extract structured data from each document:
  - Write your own extraction prompt specifying the fields to extract and the expected output format
  - Test the prompt on a single document and evaluate the output against the source image
3. Parse the extracted data and organize it into a DataFrame:
  - Process all documents through the extraction pipeline
  - Build a summary DataFrame from the parsed JSON results
4. Validate extractions using a second LLM call:
  - Apply the self-reflection pattern to check the model's own extraction work
  - Evaluate whether the validation step catches real errors
5. Analyze spending patterns using the LLM:
  - Write system and user prompts for financial analysis
  - Compare the LLM's analysis to the data in your DataFrame
6. Reflection and analysis:

- Evaluate the pipeline’s accuracy, limitations, and business implications

Important: All graded cells that require you to enter code will contain regions marked with `# YOUR CODE HERE` and `# END OF YOUR CODE`. Between these lines, you’ll find the line `raise NotImplementedError("Your code is missing.")`, like so:

```
# YOUR CODE HERE
raise NotImplementedError("Your code is missing.")
# END OF YOUR CODE
```

These markers indicate exactly where you should enter your answer. Replace the line `raise NotImplementedError("Your code is missing.")` with your code.

## 1.1 Business Context

Read through the scenario below. You will be putting yourself in the shoes of a junior ML engineer (MLE) at SpendLens, a company tasked with building an automated document extraction pipeline for one of its clients.

**1. Company and Context** SpendLens is a fintech startup based in Austin that builds automation tools for business expense management. Its products help companies digitize, categorize, and analyze their vendor spending by replacing manual data entry with AI-powered extraction.

**2. Business Challenge** SpendLens’s new client is a mid-size property management company that oversees 40 rental properties across central Texas. The company’s operations team collects expense documents from vendors, contractors, and service providers, including invoices for maintenance work, receipts for supply purchases, and statements from software subscriptions. Today, a staff member opens each document, reads through it, and manually types the relevant information (vendor name, date, line items, totals) into a spreadsheet. This process is slow, error-prone, and increasingly difficult to maintain as the portfolio grows.

**3. Business Goal** SpendLens’s goal is to build a prototype pipeline that can accept expense document images, extract structured data from each one, and produce a spending summary across the batch. If successful, the pipeline would allow the client’s operations team to process documents in minutes rather than hours and reduce data entry errors.

**4. Your Role and Task** You have just joined SpendLens as a junior MLE on the Document Intelligence team. The team has been given a set of four sample expense documents from the client and asked to build and test a prototype extraction pipeline. Your job is to design the extraction logic, validate the outputs, and analyze the results.

This work requires judgment at every stage. The extraction prompt you write determines what the model looks for and how it structures the output. The validation logic you design determines whether errors get caught before the data enters a downstream system. The decisions you make about how to handle ambiguous or missing information will directly affect the quality of the client’s spending reports.

**5. Technical Focus in This Lab** This lab focuses on building a multimodal LLM pipeline:

- **Multimodal API Calls** – Sending images and text together to a vision-capable model to extract information from documents.
- **Structured Output** – Using JSON response formatting to ensure the model returns data in a consistent, parseable structure.
- **Prompt Engineering for Extraction** – Writing prompts that specify fields, formats, and edge case handling for structured data tasks.
- **LLM-Based Validation** – Applying the self-reflection pattern to have the model check its own work against the source document.
- **LLM-Assisted Analysis** – Using the model to summarize and analyze the extracted data.

## 1.2 Part 1. Load and Inspect the Documents

SpendLens has provided you with four sample expense documents. These are stored as image files in the `data/receipts/` directory. Before you send anything to an LLM, you should look at the documents yourself to understand what you are working with.

The helper function below converts an image file to base64 format, which is required for sending images to the OpenAI API. You used this same pattern in the Extract Data project.

```
[2]: # Do not remove or edit this cell

def image_to_base64(file_path: str) -> str:
    """
    Reads an image file and returns its base64 encoded string.
    """
    with open(file_path, "rb") as f:
        return base64.b64encode(f.read()).decode("utf-8")

# Path to the directory containing the receipt images
receipts_folder = os.path.join(os.getcwd(), "data", "receipts")
```

**Task:** Use `os.listdir()` to list all files in the `receipts_folder` directory. Save the result to a variable called `image_files`. Then sort the list so the files are in a consistent order.

```
[3]: ### BEGIN SOLUTION
image_files = sorted(os.listdir(receipts_folder))
image_files
### END SOLUTION
```

```
[3]: ['receipt_01.png', 'receipt_02.png', 'receipt_03.png', 'receipt_04.png']
```

**Task:** Display the number of documents in your dataset.

```
[4]: ### BEGIN SOLUTION
print(len(image_files))
### END SOLUTION
```

Now let's display all of the images so you can see what the documents look like. Take a moment to look at each one and note the differences in format, layout, and legibility.

```
[5]: # Do not remove or edit this cell
for filename in image_files:
    file_path = os.path.join(receipts_folder, filename)
    print(f"\n--- {filename} ---")
    display(IPIImage(filename=file_path, width=400))
```

--- receipt\_01.png ---

LONE STAR REPAIR CO.

Commercial & Residential Maintenance  
7203 Mockingbird Ln, Austin, TX 78745  
(230) 555-0122 | lonestarrepair@email.com  
License #TX-CONT-88241

INVOICE

Invoice #: LSR-2026-0391  
Date: 2026-03-14  
Due Date: 2026-04-14  
Terms: Net 30

BILL TO  
Pinnacle Property Management LLC  
Attn: Operations Dept.  
8810 Research Blvd, Suite 215  
Austin, TX 78758

| DESCRIPTION                                                                            | QTY | UNIT | UNIT PRICE | AMOUNT   |
|----------------------------------------------------------------------------------------|-----|------|------------|----------|
| HVAC Filter Replacement & System Inspection<br>Unit 14B, 2118 Webberville Rd           | 1   | Job  | \$185.00   | \$185.00 |
| Plumbing Repair — Kitchen Faucet<br>Unit 7A, 2118 Webberville Rd                       | 2   | Hrs  | \$95.00    | \$190.00 |
| Parts & Materials (faucet cartridge, supply lines)<br>See attached itemized parts list | 1   | Lot  | \$62.50    | \$62.50  |
| Exterior Fence Post Reset (3 posts)<br>1405 Rundberg Ln — storm damage                 | 3   | Post | \$75.00    | \$225.00 |
| Subtotal                                                                               |     |      |            | \$662.50 |
| Tax (8.25%)                                                                            |     |      |            | \$54.66  |
| Total Due                                                                              |     |      |            | \$717.16 |

Payment Method: Credit Card  
Visa ending in 4821 | Charged on 2026-03-14  
Thank you for your business. Please retain for your records.



--- receipt\_02.png ---

**CAPITOL HARDWARE**  
YOUR LOCAL SUPPLY STORE  
6301 N Lamar Blvd, Austin TX 78752  
Tel: (230) 555-0247

-----

Date: 2026-02-07  
Time: 10:42 AM  
Trans #: TXN-88340-B  
Cashier: MARIE  
Register: REG-03

-----

|                              |         |
|------------------------------|---------|
| 3/8" Drill Bit Set (15pc)    | \$24.99 |
| SKU: 004821-HW               |         |
| Utility Knife Set (3pc)      | \$14.49 |
| SKU: 009134-TL               |         |
| Painter's Tape 1.5" (3-pack) | \$17.98 |
| SKU: 003377-PL x3            |         |
| PVC Pipe Cement (oz)         | \$8.75  |
| SKU: 007821-PL               |         |
| Paint Roller Kit 9"          | \$11.29 |
| SKU: 002200-PT               |         |
| Drop Cloth 9x12 Canvas       | \$18.50 |
| SKU: 002218-PT               |         |

-----

|                    |                 |
|--------------------|-----------------|
| Subtotal (6 items) | \$96.00         |
| Tax (8.25%)        | \$7.92          |
| <b>TOTAL</b>       | <b>\$103.92</b> |

-----



8 8 3 4 0 2 2 0 4 4 0 7 1

Returns accepted within 30 days with receipt.  
No returns on cut lumber or mixed paint.

Thank you for shopping Capitol Hardware!

--- receipt\_03.png ---

**Invoice #PS-2026-01188**

Issue Date: January 31, 2026

Period: Feb 1 – Feb 28, 2026

**FROM****PropSync Technologies, Inc.**

3601 S Congress Ave, Suite 200

Austin, TX 78704

billing@propsync.io

**BILL TO****Pinnacle Property Management LLC**

8810 Research Blvd, Suite 215

Austin, TX 78758

ops@pinnaclePM.com

| DESCRIPTION                                                                                 | QTY | UNIT PRICE | AMOUNT   |
|---------------------------------------------------------------------------------------------|-----|------------|----------|
| PropSync Professional Plan<br>Property management platform — up to 50 units. Feb 2026.      | 1   | \$249.00   | \$249.00 |
| Maintenance Request Module Add-on<br>Unlimited work orders, vendor portal access. Feb 2026. | 1   | \$59.00    | \$59.00  |
| Document Storage — Extra 50GB<br>Overage storage for document archive. Feb 2026.            | 1   | \$12.00    | \$12.00  |

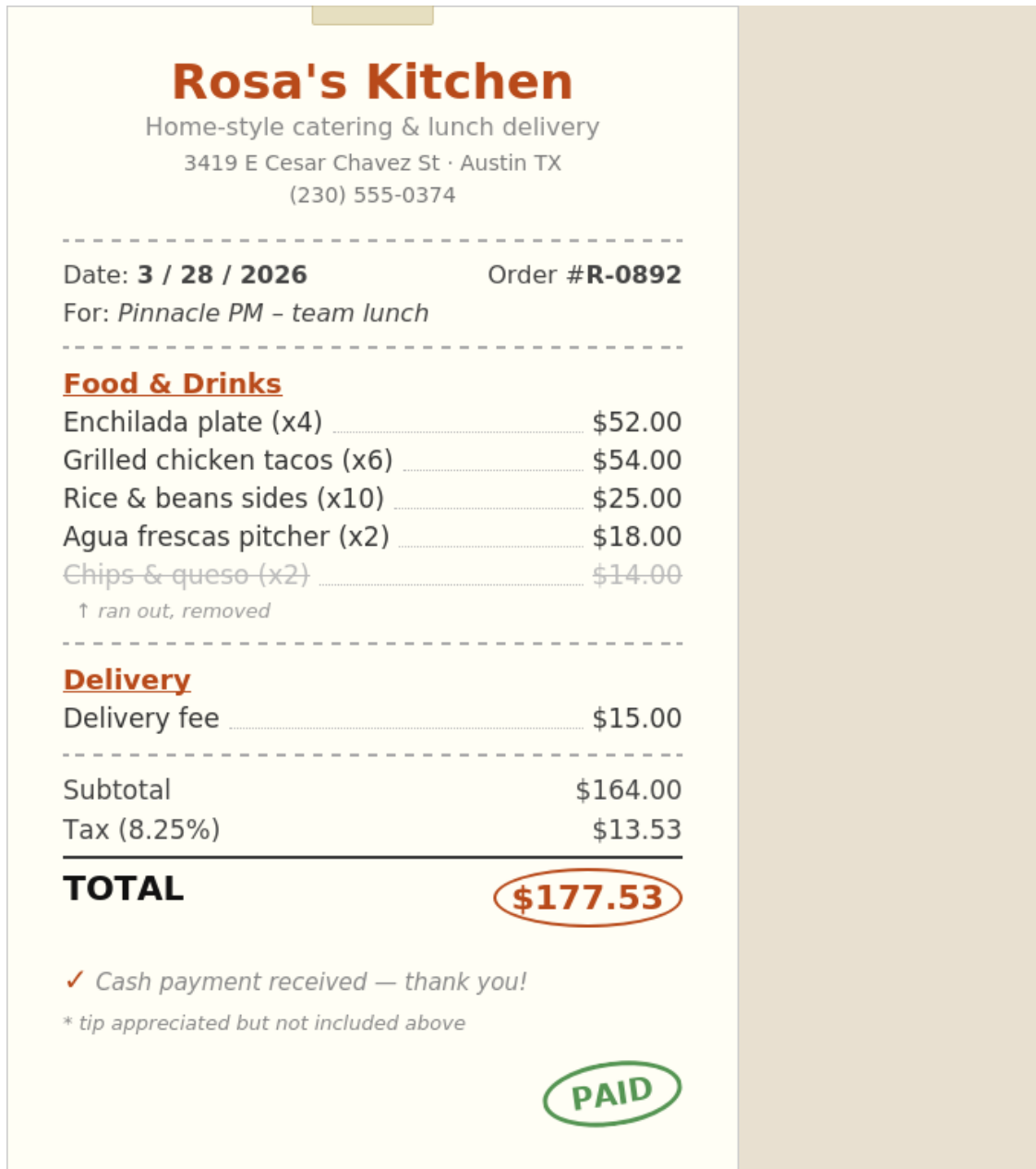
Subtotal \$320.00

Tax *Software services exempt (TX §151.3101)***Total Charged \$320.00****Payment:** Visa ending in 4821 · Auto-charged on 2026-01-31**PAID**This invoice was automatically generated. For billing questions contact [billing@propsync.io](mailto:billing@propsync.io).

Software services are exempt from Texas sales tax under TX Tax Code §151.3101. No tax has been applied.

PropSync Technologies, Inc. | EIN: 47-8821034

--- receipt\_04.png ---



**Task:** Convert all images to base64 and store them in a dictionary where the keys are filenames and the values are base64 strings. Save the result to a variable called `image_data`.

*Tip:* Loop through `image_files`, build the full file path using `os.path.join()`, and call the `image_to_base64()` function for each file.

```
[6]: ### BEGIN SOLUTION
# --- Solution (similar file-iteration pattern in Build a Transcript App
    ↪activity) ---
image_data = {}
```



```

for filename in image_files:
    file_path = os.path.join(receipts_folder, filename)
    image_data[filename] = image_to_base64(file_path)

print(f"Converted {len(image_data)} images to base64.")
### END SOLUTION

```

Converted 4 images to base64.

### 1.3 Part 2. Design Your Extraction Prompt

In the Extract Data project, the extraction prompt was provided to you. Now it is your turn to write one.

SpendLens needs the following fields extracted from each document:

- **vendor\_name**: The name of the business or vendor
- **transaction\_date**: The date of the transaction (use YYYY-MM-DD format)
- **items**: A list of items or services. Each item should include a **description** and a **price**.
- **tax\_amount**: The tax charged, if visible
- **total\_amount**: The final total
- **payment\_method**: How the bill was paid (e.g., “Credit Card”, “Cash”), if visible
- **spending\_category**: Classify the expense into one of these categories: “Office Supplies”, “Food & Dining”, “Software & Technology”, “Travel”, “Professional Services”, or “Other”

Your prompt should also instruct the model on what to do when a field is not present in the document (use `null`).

**Task:** Create a variable called `extraction_prompt` and assign it a prompt that instructs the model to extract the fields listed above from a document image. Your prompt should specify the expected output format (JSON), list every field with a description, and explain how to handle missing values.

*Tip:* Use triple quotation marks (`"""`) to write a multi-line string.

```

[7]: ### BEGIN SOLUTION
# --- Solution (fellows saw a similar extraction prompt provided in the Extract
↳Data from Images project) ---
extraction_prompt = """
Extract the following information from the document image and return it as a
↳JSON object:

- "vendor_name": The name of the business or vendor.
- "transaction_date": The date of the transaction in YYYY-MM-DD format.
- "items": A list of items or services. Each item should be an object with:
    - "description": A brief description of the item or service.
    - "price": The price for that item as a number.
- "tax_amount": The total tax charged, as a number. Use null if not visible.
- "total_amount": The final total amount, as a number.
- "payment_method": How the bill was paid (e.g., "Credit Card", "Cash"). Use
↳null if not visible.

```

```

- "spending_category": Classify this expense into exactly one of these
  categories:
    "Office Supplies", "Food & Dining", "Software & Technology", "Travel",
    "Professional Services", or "Other".

If a field is not present or cannot be determined from the document, use null.
Return only the JSON object with no additional text.
"""
### END SOLUTION

```

Before processing all four documents, let's test your prompt on a single image to make sure it works. The cell below sends the first document to the API using your extraction prompt.

```

[8]: # Do not remove or edit this cell

# Get the first image
first_filename = image_files[0]
first_image_b64 = image_data[first_filename]

# Make the API call
test_response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {
            "role": "user",
            "content": [
                {"type": "text", "text": extraction_prompt},
                {"type": "image_url", "image_url": {"url": f"data:image/png;
↳base64,{first_image_b64}"}}
            ]
        }
    ],
    response_format={"type": "json_object"},
    max_tokens=1000
)

# Display the result
test_result = test_response.choices[0].message.content
print(f"Extraction result for {first_filename}:")
print("=" * 60)
print(test_result)

```

Extraction result for receipt\_01.png:

```

=====
{
  "vendor_name": "Lone Star Repair Co.",
  "transaction_date": "2026-03-14",

```

```

"items": [
  {
    "description": "HVAC Filter Replacement & System Inspection",
    "price": 185.00
  },
  {
    "description": "Plumbing Repair - Kitchen Faucet",
    "price": 190.00
  },
  {
    "description": "Parts & Materials (faucet cartridge, supply lines)",
    "price": 62.50
  },
  {
    "description": "Exterior Fence Post Reset (3 posts)",
    "price": 225.00
  }
],
"tax_amount": 54.66,
"total_amount": 717.16,
"payment_method": "Credit Card",
"spending_category": "Professional Services"
}

```

**Task:** Look at the output above and compare it to the original image. In the markdown cell below, answer the following:

1. Did the model extract all of the fields you specified?
2. Are the extracted values accurate based on what you see in the image?
3. Is there anything you would change about your prompt based on this test?

<Double click this Markdown cell to make it editable, and record your findings here.>

## 1.4 Part 3. Extract Data from All Documents

Now that you have tested your prompt, it is time to process all four documents. You will loop through each image, send it to the API, and collect the results.

**Task:** Write a loop that iterates through every item in `image_data` (the dictionary you created in Part 1). For each image, make an API call using your `extraction_prompt`, extract the response text, and store it in the dictionary `raw_results` with the filename as the key. Print each result as it comes back so you can monitor progress.

*Tip:* Use the same API call structure shown in the test cell above. The dictionary `image_data` has filenames as keys and base64 strings as values. You can loop through it using `image_data.items()`.

```

[9]: # Do not remove or edit the line below:
raw_results = {}

### BEGIN SOLUTION

```

```

# --- Solution (multimodal content list + response_format from Extract Data,
↳from Images project; loop pattern from Build a Transcript App) ---
for filename, img_b64 in image_data.items():
    print(f"Processing: {filename}...")

    response = client.chat.completions.create(
        model="gpt-4o",
        messages=[
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": extraction_prompt},
                    {"type": "image_url", "image_url": {"url": f"data:image/png;
↳base64,{img_b64}"}}
                ]
            },
        ],
        response_format={"type": "json_object"},
        max_tokens=1000
    )

    raw_results[filename] = response.choices[0].message.content

    print(f"Result for {filename}:")
    print(raw_results[filename])
    print()
### END SOLUTION

```

Processing: receipt\_01.png...

Result for receipt\_01.png:

```

{
  "vendor_name": "Lone Star Repair Co.",
  "transaction_date": "2026-03-14",
  "items": [
    {
      "description": "HVAC Filter Replacement & System Inspection",
      "price": 185.00
    },
    {
      "description": "Plumbing Repair - Kitchen Faucet",
      "price": 190.00
    },
    {
      "description": "Parts & Materials (faucet cartridge, supply lines)",
      "price": 62.50
    },
    {

```

```

        "description": "Exterior Fence Post Reset (3 posts)",
        "price": 225.00
    }
],
"tax_amount": 54.66,
"total_amount": 717.16,
"payment_method": "Credit Card",
"spending_category": "Professional Services"
}

```

Processing: receipt\_02.png...

Result for receipt\_02.png:

```

{
  "vendor_name": "Capitol Hardware",
  "transaction_date": "2026-02-07",
  "items": [
    {
      "description": "3/8\" Drill Bit Set (15pc)",
      "price": 24.99
    },
    {
      "description": "Utility Knife Set (3pc)",
      "price": 14.49
    },
    {
      "description": "Painter's Tape 1.5\" (3-pack)",
      "price": 17.98
    },
    {
      "description": "PVC Pipe Cement (oz)",
      "price": 8.75
    },
    {
      "description": "Paint Roller Kit 9\"",
      "price": 11.29
    },
    {
      "description": "Drop Cloth 9x12 Canvas",
      "price": 18.50
    }
  ],
  "tax_amount": 7.92,
  "total_amount": 103.92,
  "payment_method": null,
  "spending_category": "Office Supplies"
}

```

Processing: receipt\_03.png...

Result for receipt\_03.png:

```
{
  "vendor_name": "PropSync Technologies, Inc.",
  "transaction_date": "2026-01-31",
  "items": [
    {
      "description": "PropSync Professional Plan",
      "price": 249.00
    },
    {
      "description": "Maintenance Request Module Add-on",
      "price": 59.00
    },
    {
      "description": "Document Storage - Extra 50GB",
      "price": 12.00
    }
  ],
  "tax_amount": null,
  "total_amount": 320.00,
  "payment_method": "Visa ending in 4821",
  "spending_category": "Software & Technology"
}
```

Processing: receipt\_04.png...

Result for receipt\_04.png:

```
{
  "vendor_name": "Rosa's Kitchen",
  "transaction_date": "2026-03-28",
  "items": [
    {
      "description": "Enchilada plate (x4)",
      "price": 52.00
    },
    {
      "description": "Grilled chicken tacos (x6)",
      "price": 54.00
    },
    {
      "description": "Rice & beans sides (x10)",
      "price": 25.00
    },
    {
      "description": "Agua frescas pitcher (x2)",
      "price": 18.00
    },
    {
      "description": "Delivery fee",

```

```

        "price": 15.00
    }
],
"tax_amount": 13.53,
"total_amount": 177.53,
"payment_method": "Cash",
"spending_category": "Food & Dining"
}

```

**Parse the JSON Results** The API returns JSON as a string. The code below parses each string into a Python dictionary so you can work with the data programmatically.

```

[10]: # Do not remove or edit this cell

parsed_results = {}

for filename, raw_json in raw_results.items():
    try:
        parsed = json.loads(raw_json)
        parsed_results[filename] = parsed
        print(f"Successfully parsed: {filename}")
    except json.JSONDecodeError as e:
        print(f"Error parsing {filename}: {e}")
        parsed_results[filename] = {"error": str(e)}

print(f"\nParsed {len(parsed_results)} documents.")

```

```

Successfully parsed: receipt_01.png
Successfully parsed: receipt_02.png
Successfully parsed: receipt_03.png
Successfully parsed: receipt_04.png

```

Parsed 4 documents.

**Build a Summary DataFrame** Now let's organize the extracted data into a pandas DataFrame. This will make it easier to review the results and spot any issues.

**Task:** Create a list called `rows` by looping through `parsed_results`. For each document, create a dictionary with the following keys: "filename", "vendor\_name", "transaction\_date", "total\_amount", "tax\_amount", "payment\_method", "spending\_category", and "num\_items" (the number of items in the "items" list). Append each dictionary to `rows`.

Then create a DataFrame called `df` from `rows`.

*Tip:* Use the `.get()` method to safely access dictionary keys, in case a field is missing. For example, `parsed.get("vendor_name", None)` returns `None` if the key does not exist. For `num_items`, use `len()` on the items list, but make sure to handle the case where the list might not exist.

```
[11]: ### BEGIN SOLUTION
# --- Solution (DataFrame construction from Units 1-2; .get() on dicts is new,
      ↳but explained in the tip above) ---
rows = []

for filename, parsed in parsed_results.items():
    items_list = parsed.get("items", [])

    row = {
        "filename": filename,
        "vendor_name": parsed.get("vendor_name", None),
        "transaction_date": parsed.get("transaction_date", None),
        "total_amount": parsed.get("total_amount", None),
        "tax_amount": parsed.get("tax_amount", None),
        "payment_method": parsed.get("payment_method", None),
        "spending_category": parsed.get("spending_category", None),
        "num_items": len(items_list) if items_list else 0
    }

    rows.append(row)

df = pd.DataFrame(rows)
### END SOLUTION
```

**Task:** Display the full DataFrame.

```
[12]: ### BEGIN SOLUTION
df
### END SOLUTION
```

```
[12]:
```

|   | filename       | vendor_name                 | transaction_date | total_amount | \ |
|---|----------------|-----------------------------|------------------|--------------|---|
| 0 | receipt_01.png | Lone Star Repair Co.        | 2026-03-14       | 717.16       |   |
| 1 | receipt_02.png | Capitol Hardware            | 2026-02-07       | 103.92       |   |
| 2 | receipt_03.png | PropSync Technologies, Inc. | 2026-01-31       | 320.00       |   |
| 3 | receipt_04.png | Rosa's Kitchen              | 2026-03-28       | 177.53       |   |

|   | tax_amount | payment_method      | spending_category     | num_items |
|---|------------|---------------------|-----------------------|-----------|
| 0 | 54.66      | Credit Card         | Professional Services | 4         |
| 1 | 7.92       | None                | Office Supplies       | 6         |
| 2 | NaN        | Visa ending in 4821 | Software & Technology | 3         |
| 3 | 13.53      | Cash                | Food & Dining         | 5         |

## 1.5 Part 4. Validate Extractions

Extraction is only useful if it is accurate. In a production pipeline, you would want a quality-check step that flags potential errors before the data enters a downstream system.

You are going to use the same multimodal model to validate its own work. The idea is to send



the original image *and* the extracted data back to the model and ask it to check for discrepancies. This applies the self-reflection pattern you practiced earlier in this unit, but in a data extraction context.

Specifically, the validation prompt should ask the model to check: - Whether the total matches the sum of item prices plus tax - Whether the vendor name and date are correct - Whether any line items were missed or duplicated - Whether the spending category is appropriate

**Task:** Create a variable called `validation_prompt_template` that contains a validation prompt. This prompt will be combined with the extracted data for each document using an f-string. Use the placeholder `{extracted_json}` where the extracted data should appear.

Your prompt should: 1. Explain the task (check the extracted data against the original image) 2. Include the extracted data (using the `{extracted_json}` placeholder) 3. List the specific checks to perform (the four items listed above) 4. Instruct the model to output a brief assessment: either “PASS” if everything looks correct, or a description of each issue found

*Note:* Because this template contains curly braces for the f-string placeholder `{extracted_json}`, you should define it as a regular string now and use `.format()` later, rather than using an f-string directly. For example:

```
validation_prompt_template = "Check the following data: {extracted_json}"
```

```
[13]: ### BEGIN SOLUTION
# --- Solution (self-reflection pattern from Design and Test Self-Reflection,
#       ↳ Prompts project; .format() is new but explained in the note above) ---
validation_prompt_template = """
You are a data quality auditor. Compare the extracted data below against the
↳ original
document image and check for errors.

Extracted data:
{extracted_json}

Perform the following checks:
1. Does the total_amount match the sum of all item prices plus tax_amount?
2. Is the vendor_name correct based on what appears in the image?
3. Is the transaction_date correct?
4. Were any line items missed or duplicated?
5. Is the spending_category appropriate for the type of purchase shown?

If everything looks correct, respond with "PASS" and a one-sentence
↳ confirmation.
If you find any issues, describe each one briefly.
"""
### END SOLUTION
```

Now let's run the validation on each document. The cell below loops through all documents, formats the validation prompt with each document's extracted data, and sends both the prompt and the original image to the model.

```
[14]: # Do not remove or edit this cell

validation_results = {}

for filename in parsed_results:
    # Format the extracted data as a readable JSON string
    extracted_json = json.dumps(parsed_results[filename], indent=2)

    # Build the validation prompt with the extracted data inserted
    filled_prompt = validation_prompt_template.
    ↪format(extracted_json=extracted_json)

    # Send the original image and the validation prompt to the model
    val_response = client.chat.completions.create(
        model="gpt-4o",
        messages=[
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": filled_prompt},
                    {"type": "image_url", "image_url": {"url": f"data:image/png;
    ↪base64,{image_data[filename]}"}}
                ]
            }
        ],
        max_tokens=500
    )

    result = val_response.choices[0].message.content
    validation_results[filename] = result

    print(f"\n{'=' * 60}")
    print(f"Validation for: {filename}")
    print(f"{'=' * 60}")
    display(Markdown(result))
```

```
=====
Validation for: receipt_01.png
=====
```

#### 1. Total Amount Check:

- Sum of item prices:  $\$185.00 + \$190.00 + \$62.50 + \$225.00 = \$662.50$
- Total with tax:  $\$662.50 + \$54.66 = \$717.16$
- **Correct:** The total amount matches.

#### 2. Vendor Name Check:

- Extracted: "Lone Star Repair Co."
- Image: "Lone Star Repair Co."

- **Correct:** The vendor name matches.
3. **Transaction Date Check:**
    - Extracted: “2026-03-14”
    - Image: “2026-03-14”
    - **Correct:** The transaction date matches.
  4. **Line Items Check:**
    - All line items in the extracted data match the image.
    - **Correct:** No line items are missed or duplicated.
  5. **Spending Category Check:**
    - “Professional Services” is appropriate for maintenance and repair services.
    - **Correct:** The spending category is appropriate.

**PASS:** All data has been accurately extracted and verified against the original document.

=====

Validation for: receipt\_02.png

=====

1. **Total Amount Check:** The sum of item prices (\$96.00) plus the tax amount (\$7.92) matches the total amount of \$103.92. **Correct.**
2. **Vendor Name:** The vendor name “Capitol Hardware” matches the name in the document. **Correct.**
3. **Transaction Date:** The transaction date “2026-02-07” is the same in both the extracted data and the document. **Correct.**
4. **Line Items:** All six line items are present, with no duplicates or omissions. **Correct.**
5. **Spending Category:** The spending category “Office Supplies” might not be appropriate, as these items appear to be hardware or home improvement supplies. **Potential Issue.**

Please verify the spending category for accuracy.

=====

Validation for: receipt\_03.png

=====

1. **Total Amount Check:** PASS - The total amount (\$320.00) matches the sum of the item prices since the tax is exempt.
2. **Vendor Name Check:** PASS - The vendor name “PropSync Technologies, Inc.” is correct.
3. **Transaction Date Check:** PASS - The transaction date “2026-01-31” is correct.
4. **Line Item Check:** PASS - All line items are listed once, with no duplicates or omissions.
5. **Spending Category Check:** PASS - “Software & Technology” is appropriate for this purchase.

**PASS -** All extracted data is accurate and matches the original document.

=====

Validation for: receipt\_04.png

=====

1. **Total Amount Check:** The sum of the item prices plus the tax amount is correct (\$177.53).
2. **Vendor Name Check:** The vendor name “Rosa’s Kitchen” is correct.
3. **Transaction Date Check:** The transaction date “2026-03-28” is correct.
4. **Line Items Check:** No line items were missed or duplicated. The removed item “Chips & queso” is correctly not included.
5. **Spending Category Check:** The spending category “Food & Dining” is appropriate.

**PASS:** All the extracted data is accurate and complete.

**Task:** Review the validation results above. In the markdown cell below, answer the following:

1. Did the validation step catch any real errors in the extracted data?
2. Did it flag anything as an issue that was not actually a problem (a false positive)?
3. What does this tell you about using an LLM to check its own work? Is it a reliable quality control method?

<Double click this Markdown cell to make it editable, and record your findings here.>

## 1.6 Part 5. Analyze Spending Patterns

SpendLens’s clients do not just want digitized receipts. They want insights. Now that you have structured data from all four documents, you will use the LLM to analyze spending patterns across the batch.

The cell below combines all of your extracted data into a single JSON string that you can pass to the model as context.

```
[15]: # Do not remove or edit this cell

all_extracted_data = json.dumps(parsed_results, indent=2)

print("Combined extracted data (preview):")
print(all_extracted_data[:500])
print("\n...")
```

```
Combined extracted data (preview):
{
  "receipt_01.png": {
    "vendor_name": "Lone Star Repair Co.",
    "transaction_date": "2026-03-14",
    "items": [
      {
        "description": "HVAC Filter Replacement & System Inspection",
        "price": 185.0
      },
      {
```

```

    "description": "Plumbing Repair \u2013 Kitchen Faucet",
    "price": 190.0
},
{
    "description": "Parts & Materials (faucet cartridge, supply lines)",
    "price": 62.5
},
{
    "description": "Exterior Fence Pos

```

...

**Task:** Write two variables:

1. `analysis_system_prompt`: A system prompt that defines the model's role. Position the model as a financial analyst working for SpendLens. It should produce clear, professional summaries.
2. `analysis_user_prompt`: A user prompt that includes the extracted data (use an f-string with `{all_extracted_data}`) and asks the model to:
  - Summarize total spending across all documents
  - Break down spending by category
  - Identify the single largest expense
  - Flag anything unusual or noteworthy (e.g., missing information, high-value transactions)

```

[16]: ### BEGIN SOLUTION
# --- Solution (system prompt from Personalize with System Prompts project;
    ↪f-string data injection from Chat With Your Document and RAG Chatbot
    ↪projects) ---
analysis_system_prompt = """You are a financial analyst working for SpendLens.
You produce clear, professional spending summaries for clients.
Be specific with numbers and organize your analysis clearly."""

analysis_user_prompt = f"""Here is the extracted expense data from a batch of
    ↪documents
submitted by a property management client:

{all_extracted_data}

Please provide the following analysis:
1. Summarize total spending across all documents.
2. Break down spending by category.
3. Identify the single largest expense.
4. Flag anything unusual or noteworthy, such as missing information or
    high-value transactions that may warrant review."""
### END SOLUTION

```

Now let's run the analysis.

```
[17]: # Do not remove or edit this cell
```

```
analysis_response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": analysis_system_prompt},
        {"role": "user", "content": analysis_user_prompt}
    ],
    max_tokens=1500
)

analysis_text = analysis_response.choices[0].message.content

print("Spending Analysis")
print("=" * 60)
display(Markdown(analysis_text))
```

Spending Analysis

=====

### 1. Total Spending Across All Documents:

The total spending across all four receipts amounts to \$1,318.61. This is calculated as follows:

- Receipt 01: \$717.16
- Receipt 02: \$103.92
- Receipt 03: \$320.00
- Receipt 04: \$177.53

### 2. Spending Breakdown by Category:

- **Professional Services:** \$717.16 (1 transaction)
  - Includes HVAC maintenance and plumbing repairs.
- **Office Supplies:** \$103.92 (1 transaction)
  - Comprises purchases such as drill bit sets, utility knives, and painting supplies.
- **Software & Technology:** \$320.00 (1 transaction)
  - Subscriptions and add-ons for property management software.
- **Food & Dining:** \$177.53 (1 transaction)
  - Catering from a local restaurant.

### 3. Single Largest Expense:

- The single largest expense is associated with **Professional Services**, specifically with “Lone Star Repair Co.” totaling \$717.16. This includes various maintenance activities and materials.

### 4. Unusual or Noteworthy Items:

- **Missing Payment Method:** For the transaction with “Capitol Hardware,” the payment method data is missing. This is noteworthy as it could be important for financial tracking and record-keeping.
- **High-Value Transaction:** The expense with “Lone Star Repair Co.” is a high-value trans-

action which significantly tops other expenses. It's prudent to verify that all services listed were indeed necessary and completed, given the total amount spent (\$717.16).

Overall, while most transactions appear standard for property management operations, ensuring all payment methods are documented will improve financial transparency. Additionally, reviewing the high-value maintenance service might provide insights or assurance about the efficient use of resources.

**Task:** Review the analysis above and compare it to the DataFrame you built in Part 3. In the markdown cell below, answer the following:

1. Does the model's summary match the numbers in your DataFrame? Are the totals and category breakdowns consistent with what you can verify yourself?
2. Did the model surface any insight you would not have noticed by looking at the DataFrame alone?
3. Where would you trust this analysis, and where would you want a human to double-check?

<Double click this Markdown cell to make it editable, and record your findings here.>

## 1.7 Part 6. Analysis

You have now built a complete document extraction and analysis pipeline for SpendLens. In this final section, reflect on the pipeline as a whole, both technically and in the context of the business problem.

Answer the following questions in the markdown cell below:

1. **Accuracy:** What were the biggest sources of error in your extraction pipeline? Were errors more common in certain types of documents (e.g., simple vs. complex layouts)? Where would you focus improvement efforts?
2. **Prompt design:** You wrote two prompts in this lab: one for extraction and one for validation. Which was harder to get right, and why? What did you learn about writing effective prompts for structured data tasks?
3. **Business risk:** SpendLens is considering offering this pipeline as a product feature for the property management client. Based on what you observed in this lab, what is the biggest risk of deploying this pipeline without human review? Name one type of document or scenario that would likely cause the pipeline to fail, and explain what you would recommend to the client.

<Double click this Markdown cell to make it editable, and record your findings here.>

## 1.8 Part 7. Reflection: AI Usage

1. Did you use AI tools for this lab? If yes, which ones and at what points in your work? If no, briefly explain your reasoning.
2. If you used AI, describe one specific prompt that was useful and explain why it worked. If you did not use AI, walk through one part of the lab where you had to figure something out on your own and explain how you got there.
3. How did you verify that your work was correct? What would you look for to catch a mistake, whether it came from AI or from your own reasoning?

4. What is one thing you would do differently next time, either in how you approached the lab or in how you used (or did not use) AI?

Record your findings in the cell below.

<Double click this Markdown cell to make it editable, and record your findings here.>